

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD

«¿Quién ganará? Greedy vs Dinámico vs Bruto.»

Nicolas Arellano

24 de junio de 2026

18:18

1. Entorno experimental

El entorno experimental utilizado para la ejecución y medición de los algoritmos implementados es el siguiente:

- CPU: Intel(R) Core(TM) i5-7300HQ CPU @ 2.50GHz
- Memoria RAM: 8GiB + 4 GiB DDR4, ambas a una velocidad de 2400 MT/s
- Sistema Operativo: Fedora 44 con escritorio KDE Plasma
- Visual Studio Code para la escritura de código
- Se usaron los lenguajes de programación Python 3.12.6 y C++ (g++ (GCC) 16.1.1 20260515 (Red Hat 16.1.1-2))

Los casos de prueba fueron generados automáticamente mediante el script `testcases_generator.py`, ubicado en `code/implementation/scripts/`. Se generaron casos de tres tamaños: pequeños ($n \in \{3, 5, 8\}$), medianos ($n \in \{20, 40, 80\}$) y grandes ($n \in \{100, 150, 200\}$), con 3 instancias aleatorias por tamaño. Adicionalmente, se generaron casos controlados variando individualmente cada parámetro del problema: cantidad de animes (n), minutos disponibles (M), energía disponible (E) y capítulos por anime (q), manteniendo los demás fijos.

Las mediciones de tiempo se realizaron con `chrono::high_resolution_clock` en microsegundos y la memoria se midió leyendo `/proc/self/status` (campo `VmRSS`). Los gráficos fueron generados automáticamente con el script `plot_generator.py` y se encuentran en `code/implementation/data/plots/`.

2. Resultados y Análisis

Los resultados obtenidos se presentan a continuación mediante gráficos generados automáticamente a partir de los datos producidos por los algoritmos.

Tiempo de ejecución

La Figura 1 muestra el tiempo de ejecución de fuerza bruta y los dos greedy. Se observa que greedy1 crece linealmente con n , llegando a aproximadamente 60 μs para $n = 200$, mientras que greedy2 es consistentemente más rápido, llegando a 21 μs . Fuerza bruta solo aparece para $n \in \{3, 5, 8\}$ y su tiempo es irregular dado que depende fuertemente de los valores aleatorios de M y E .

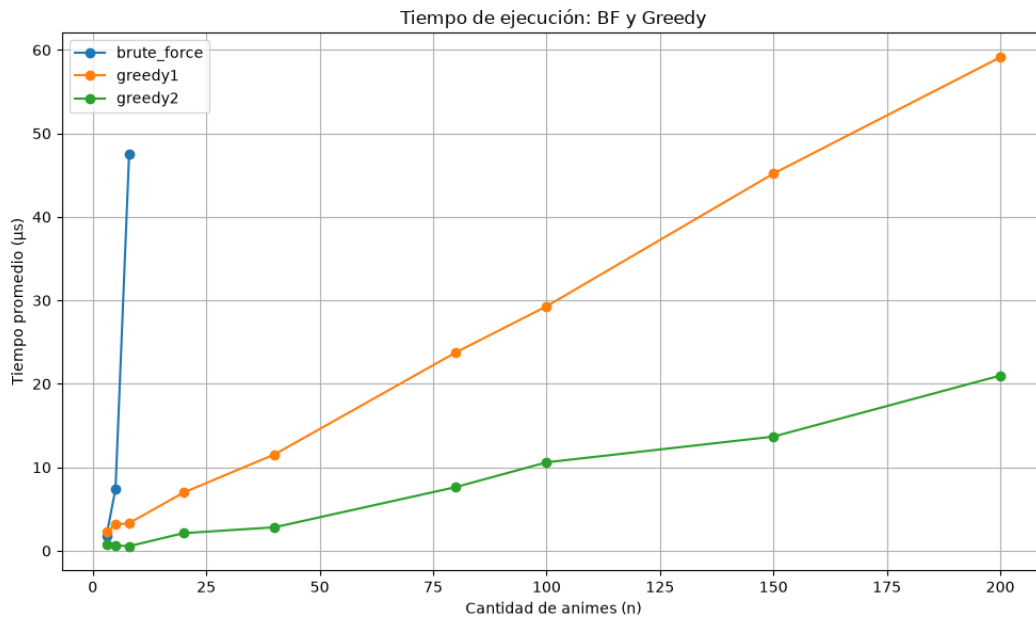


Figura 1: Tiempo de ejecución de Fuerza Bruta y Greedy según cantidad de animés.

La Figura 2 muestra el tiempo de DP de forma separada dada la diferencia de escala. Se aprecia una tendencia creciente general, aunque con variaciones producto de los valores aleatorios de M y E en cada caso, ya que la complejidad de DP es $O(n \cdot q \cdot M \cdot E)$ y los tres últimos factores varían entre instancias.

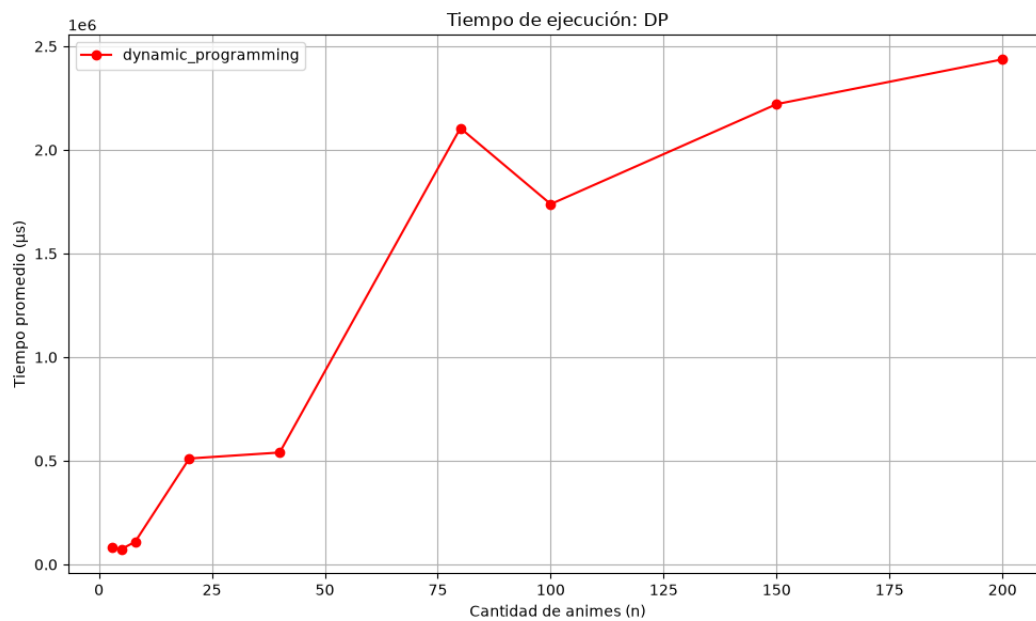


Figura 2: Tiempo de ejecución de Programación Dinámica según cantidad de animes.

Uso de memoria

La Figura 3 muestra el uso de memoria de cada algoritmo. DP utiliza aproximadamente 16.000 KB de forma constante independiente de n , lo cual se explica porque la tabla `dp[3001][501]` se declara globalmente con tamaño fijo máximo ($3001 \times 501 \times 8 \text{ bytes} \approx 12 \text{ MB}$), sin importar el tamaño real de la instancia. Los demás algoritmos mantienen un uso constante de aproximadamente 4.000 KB correspondiente a la memoria base del proceso.

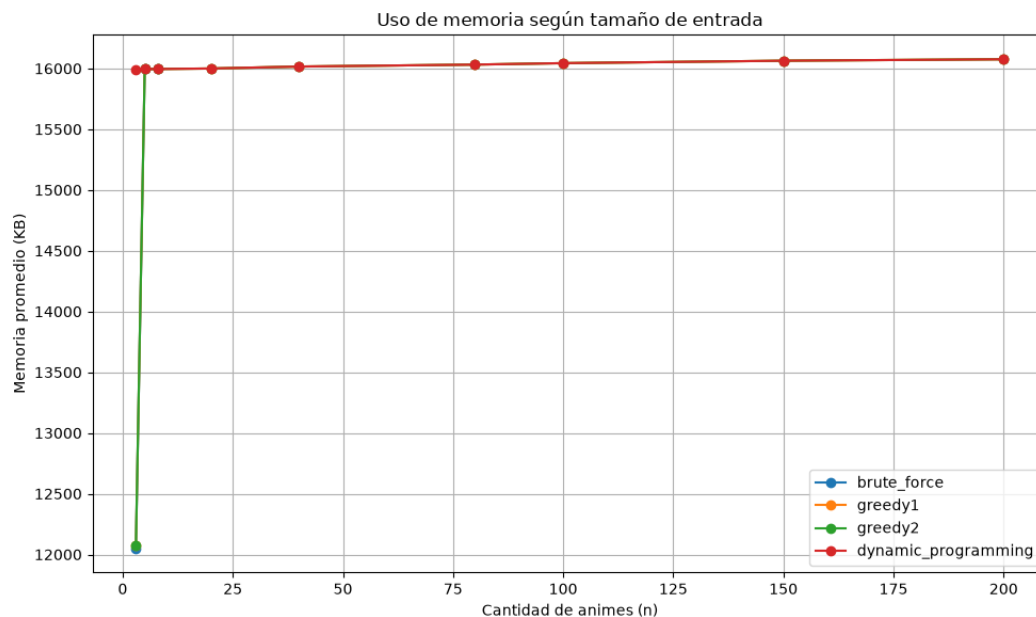


Figura 3: Uso de memoria según cantidad de animes.

Calidad de solución

Las Figuras 4 y 5 muestran el porcentaje del óptimo alcanzado por cada greedy respecto a DP. Greedy1, que ordena por bono de completación, alcanza entre un 22 % y 78 % del óptimo, con tendencia a empeorar para instancias grandes. Esto se debe a que ignorar el costo en tiempo y energía al priorizar por bono es una decisión local que no refleja bien el valor real de cada anime. Greedy2, que usa el criterio $v/(t+c)$, se desempeña considerablemente mejor, alcanzando entre un 61 % y 98 % del óptimo, lo que confirma que considerar el costo de los recursos mejora significativamente la calidad de la solución.

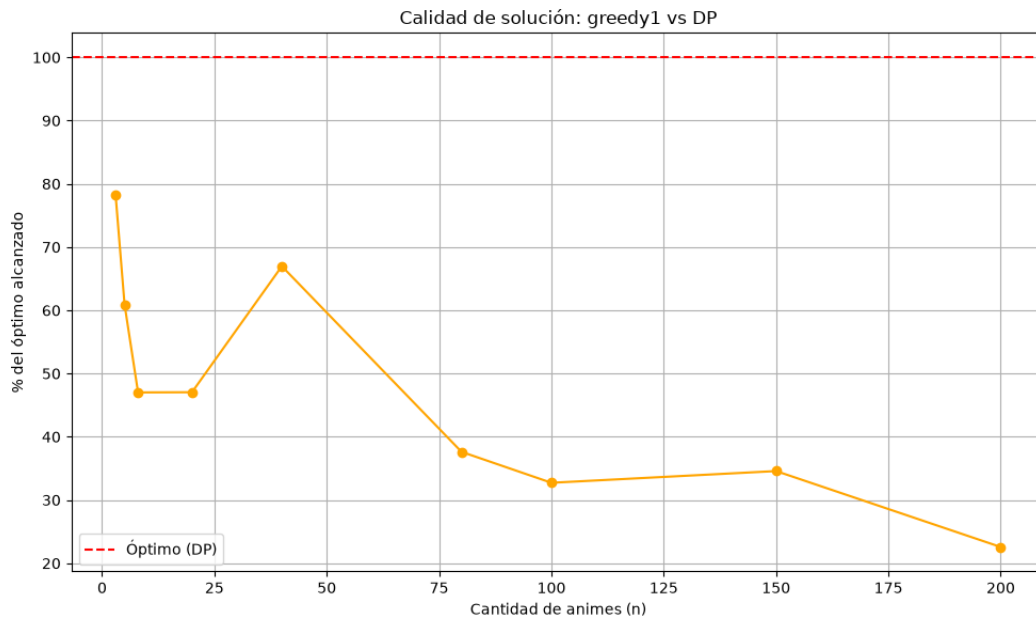


Figura 4: Porcentaje del óptimo alcanzado por Greedy1 respecto a DP.

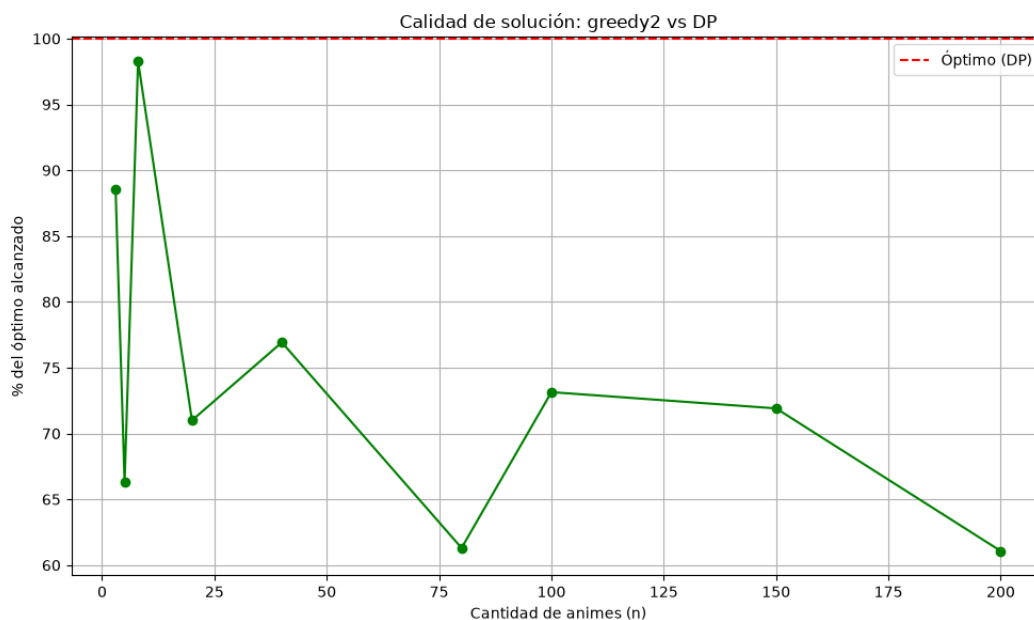


Figura 5: Porcentaje del óptimo alcanzado por Greedy2 respecto a DP.

Comportamiento al variar parámetros

Las Figuras 6, 7, 8 y 9 muestran el tiempo de ejecución variando cada parámetro de forma controlada. En todos los casos, greedy1 y greedy2 permanecen prácticamente en 0 μ s, confirmando su independencia de estos parámetros. DP en cambio crece linealmente al aumentar n , M , E y q , lo cual es consistente con su complejidad teórica $O(n \cdot q \cdot M \cdot E)$.

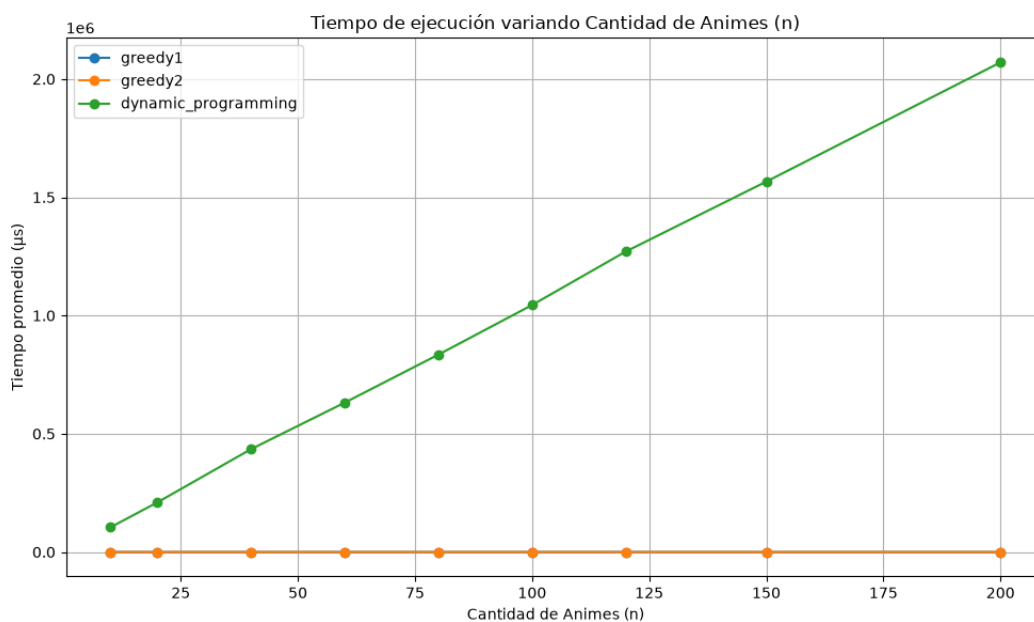


Figura 6: Tiempo de ejecución variando cantidad de anime n ($M=1500$, $E=250$ fijos).

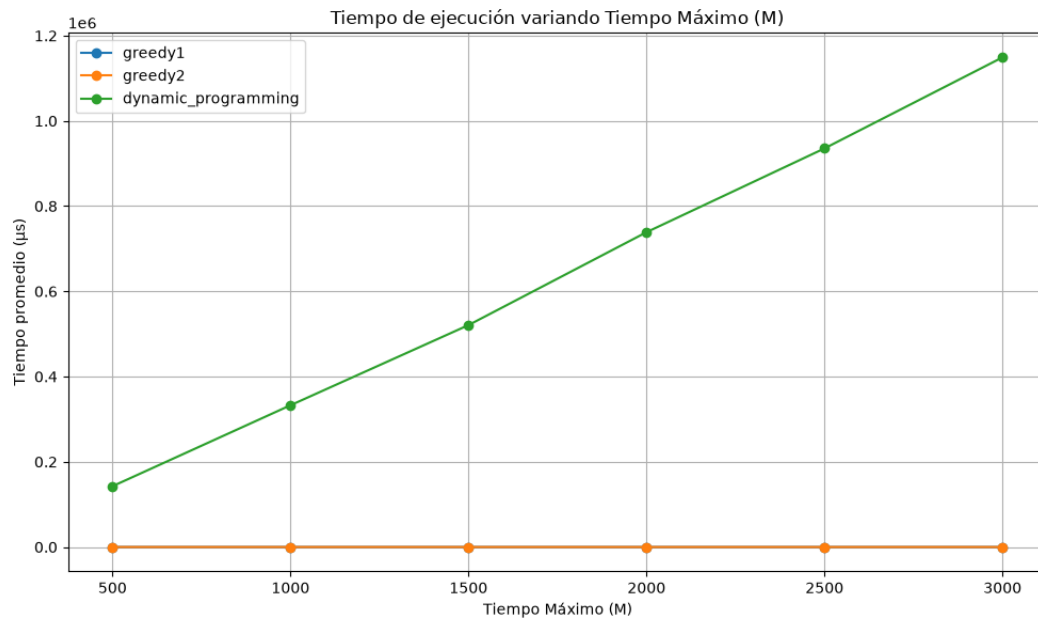


Figura 7: Tiempo de ejecución variando minutos disponibles M ($n=50$, $E=250$ fijos).

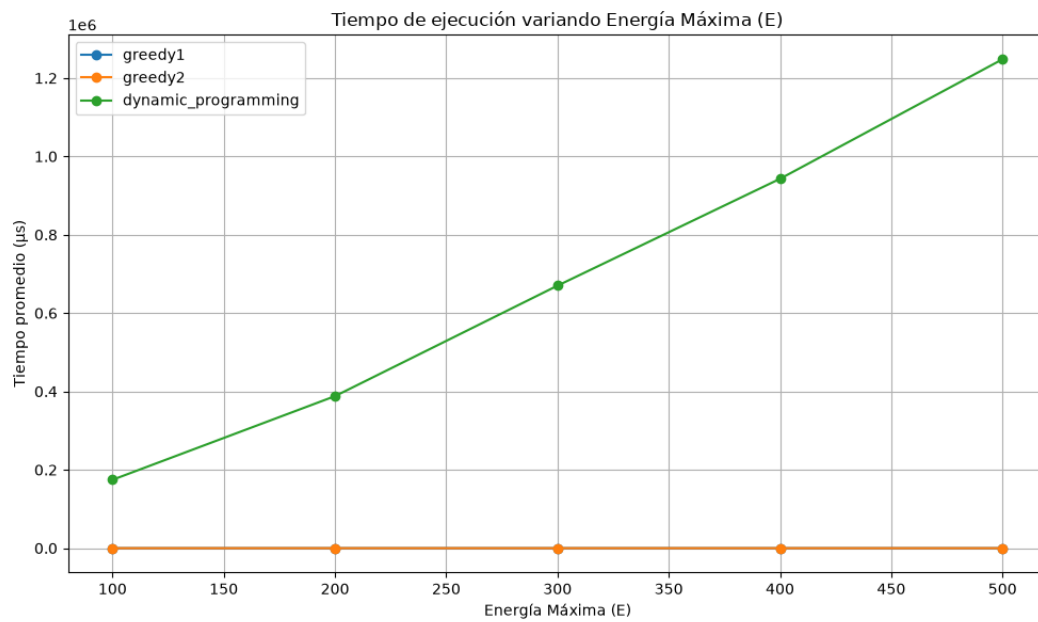


Figura 8: Tiempo de ejecución variando energía disponible E ($n=50$, $M=1500$ fijos).

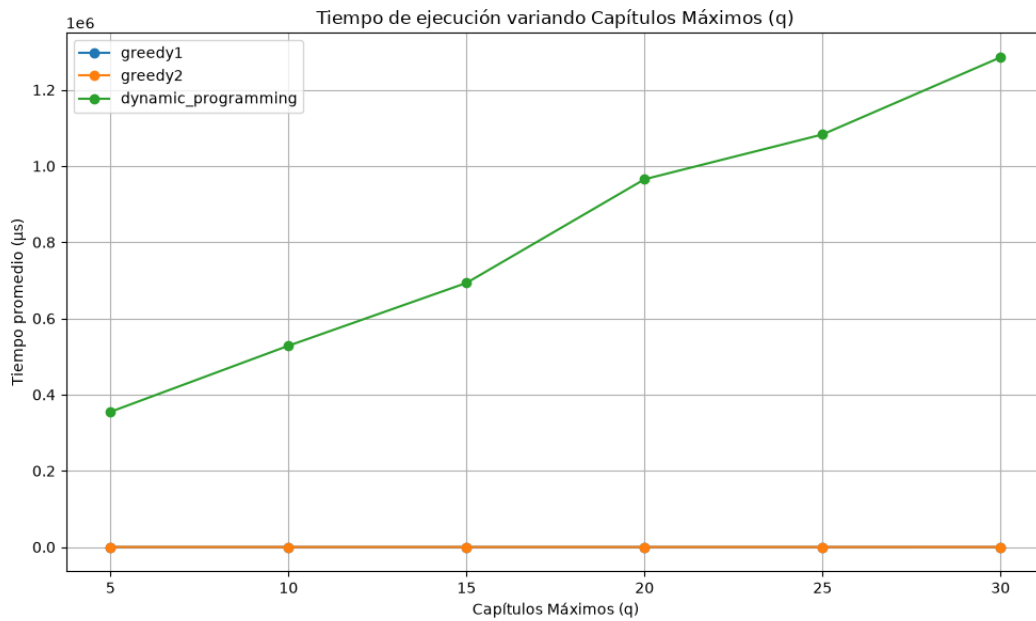


Figura 9: Tiempo de ejecución variando capítulos máximos q ($n=50$, $M=1500$, $E=250$ fijos).

3. Conclusiones

Los experimentos realizados muestran que existe un claro trade-off entre calidad de solución y eficiencia. Fuerza bruta garantiza el óptimo pero es inviable para instancias medianas o grandes. Programación dinámica también obtiene el óptimo y escala razonablemente, aunque su tiempo crece linealmente con n , M , E y q simultáneamente, lo que la hace lenta para instancias con recursos máximos. Los algoritmos greedy son extremadamente rápidos en todos los casos, pero sacrifican calidad: greedy1 alcanza solo entre un 22 % y 78 % del óptimo, mientras que greedy2 logra entre 61 % y 98 %, demostrando que un criterio local más informado como $v/(t+c)$ produce soluciones notablemente mejores sin costo computacional adicional significativo.